Closy – Kiến trúc hệ thống

Oct 20, 2026 · @tim

Closy dùng kiến trúc ba khối: ứng dụng di động theo hướng offline-first, backend Firebase và các dịch vụ ngoài (AI nhận diện trang phục, thời tiết). Công nghệ cụ thể là đề xuất, cần nhóm L01_abc xác nhận ở mục cuối.

1. Yêu cầu dẫn dắt kiến trúc

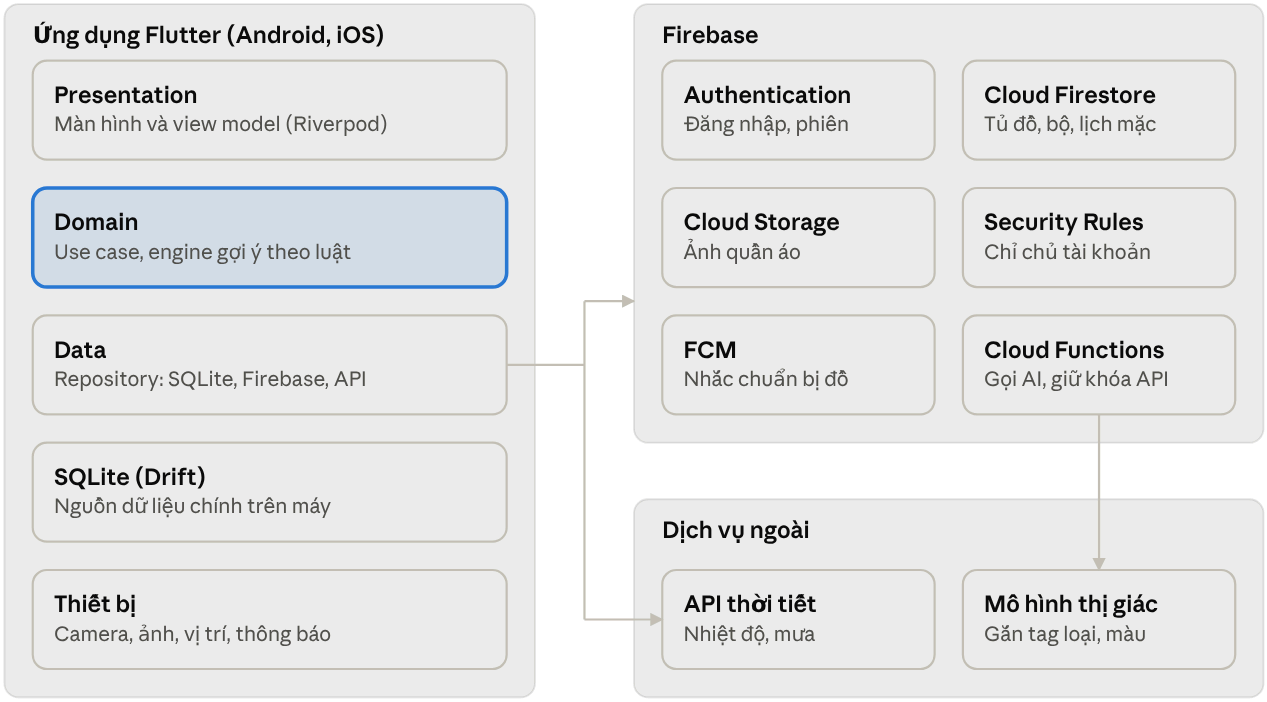
Kiến trúc phải phục vụ ba nhu cầu đã chốt trong Assignment 1: giảm thời gian lựa chọn, phối đồ phù hợp bối cảnh và tăng khả năng sử dụng lại trang phục. Mỗi nhu cầu kéo theo một yêu cầu kỹ thuật cụ thể.

Nhu cầu (Assignment 1)	Yêu cầu kỹ thuật	Hệ quả kiến trúc
Giảm thời gian lựa chọn	Mở app và có gợi ý trong vài giây, kể cả khi mạng yếu	Dữ liệu tủ đồ lưu cục bộ, engine gợi ý chạy trên máy
Phối đồ phù hợp bối cảnh (đi học, đi làm, cà phê, hẹn hò)	Mỗi món đồ có thuộc tính màu, loại, độ chỉn chu; gợi ý lọc theo hoạt động và thời tiết	Mô hình dữ liệu có tag; tích hợp API thời tiết
Tăng khả năng dùng lại đồ	Biết món nào lâu chưa mặc	Lưu lịch mặc; engine ưu tiên món ít mặc
Số hóa tủ đồ nhanh (không nhớ hết đồ)	Chụp một lần, tự gắn tag	Camera, upload ảnh, dịch vụ AI nhận diện chạy phía server

Hai bối cảnh bổ sung (dọn tủ, trước khi mua đồ mới) dùng lại cùng dữ liệu tủ đồ và lịch mặc, nên không cần thành phần mới ở giai đoạn MVP.

2. Tổng quan kiến trúc

Gợi ý chạy trên máy; mạng chỉ dùng để đồng bộ và gắn tag



kiến trúc tổng quan · app, Firebase, dịch vụ ngoài

Tầng Data ghi vào SQLite trước, rồi đồng bộ lên Firebase và gọi API thời tiết. Ảnh mới đi qua Cloud Functions đến mô hình thị giác để gắn tag, kết quả quay về app để người dùng xác nhận.

3. Quyết định công nghệ

Flutter kết hợp Firebase phù hợp nhất với nhóm bốn người: một codebase cho cả Android và iOS, không phải tự dựng và vận hành server. Nếu nhóm đã chốt stack khác, bảng dưới vẫn đúng ở mức vai trò của từng thành phần.

Hạng mục	Lựa chọn đề xuất	Lý do	Phương án thay thế
Nền tảng app	Flutter (Dart)	Một codebase cho hai hệ điều hành; dễ dựng lưới ảnh quần áo tùy biến	React Native, Kotlin/Swift native

Hạng mục	Lựa chọn đề xuất	Lý do	Phương án thay thế
Kiến trúc app	Feature-first + Clean Architecture, mẫu MVVM	Tách logic gợi ý khỏi giao diện để kiểm thử riêng	MVC, BLoC thuần
Quản lý trạng thái	Riverpod	Gọn, dễ inject repository và use case	BLoC, Provider
Lưu cục bộ	SQLite qua Drift	Truy vấn lọc theo loại, màu, lần mặc cuối	Hive, Isar
Backend	Firebase: Auth, Firestore, Storage, Cloud Functions, FCM	Đủ cho MVP, không cần vận hành server	Supabase, Node.js + PostgreSQL
Gắn tag trang phục	Mô hình thị giác gọi qua Cloud Functions; người dùng xác nhận hoặc sửa	Khóa API nằm ở server, không nằm trong app	Gắn tag thủ công hoàn toàn; ML Kit chạy trên máy
Engine gợi ý	Chấm điểm theo luật, chạy trên máy	Chạy offline, giải thích được vì sao chọn bộ này	Dùng LLM sinh bộ phối
Thời tiết	API thời tiết công khai (ví dụ Open-Meteo)	Lọc món theo nhiệt độ và mưa	Cho người dùng chọn thời tiết thủ công
Nhắc nhở	FCM và thông báo cục bộ	Nhắc chuẩn bị đồ cho hôm sau	Không có thông báo ở MVP

Ở MVP, AI chỉ gắn tag cho món đồ; quyết định phối đồ do engine theo luật đảm nhiệm. Cách này cho kết quả lặp lại được và tránh phụ thuộc mạng vào đúng lúc người dùng đang vội.

4. Kiến trúc trong ứng dụng

Ứng dụng chia theo tính năng; trong mỗi tính năng có ba tầng với quy tắc phụ thuộc một chiều: giao diện gọi use case, use case gọi repository, repository mới chạm vào cơ sở dữ liệu hoặc mạng.

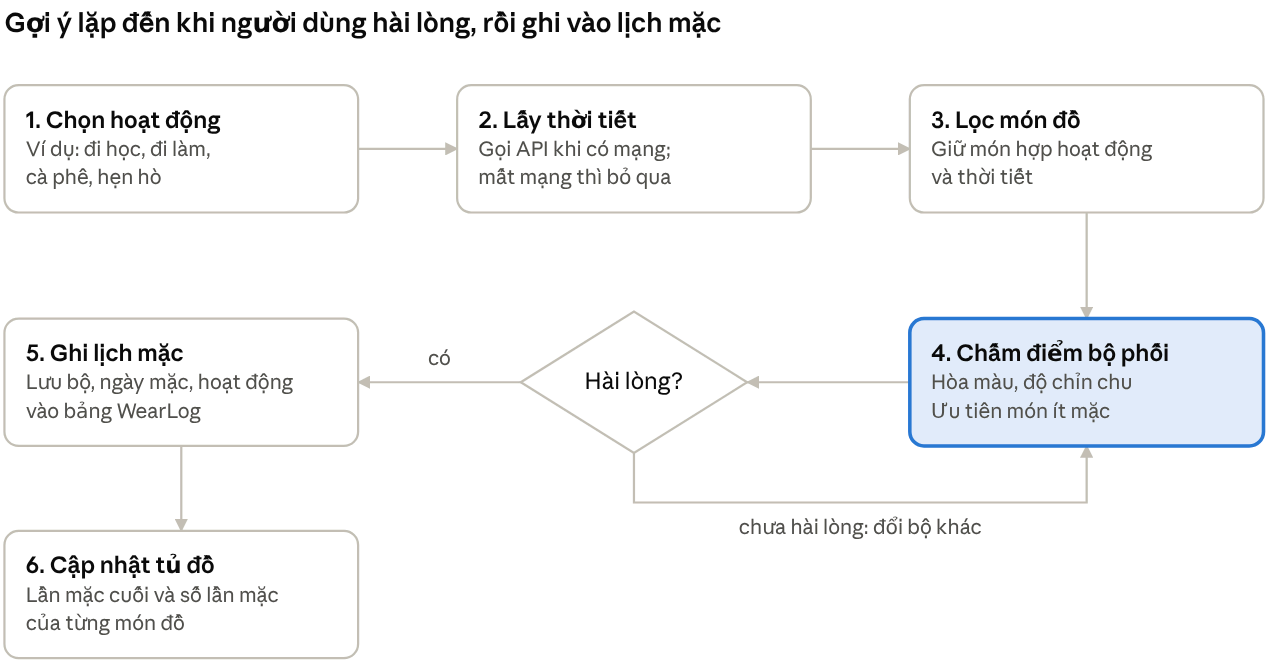
1. **Presentation**: màn hình và view model, chỉ giữ trạng thái hiển thị.
2. **Domain**: thực thể và use case (`AddGarment`, `SuggestOutfit`, `LogWear`), không phụ thuộc Flutter hay Firebase.

3. **Data**: repository ghi vào SQLite trước, rồi đồng bộ nền lên Firestore và Storage; gọi API thời tiết và Cloud Functions.

Module	Trách nhiệm	Màn hình chính
auth	Đăng ký, đăng nhập, hồ sơ và thành phố mặc định	Đăng nhập, Hồ sơ
closet	Thêm đồ bằng ảnh, gắn và sửa tag, lọc, xem, xóa	Tủ đồ, Chi tiết món đồ, Thêm đồ
outfit	Chọn hoạt động, nhận gợi ý, đổi bộ khác, lưu bộ yêu thích	Gợi ý hôm nay, Chi tiết bộ
history	Ghi lịch mặc, tìm món lâu chưa mặc	Lịch mặc, Đồ ít mặc
settings	Thông báo, quyền riêng tư, xóa dữ liệu	Cài đặt
core	Cơ sở dữ liệu, mạng, theme, xử lý lỗi, dependency injection	Không có màn hình

Khi đồng bộ có xung đột giữa hai thiết bị, bản ghi sửa sau cùng được giữ lại.

5. Luồng gợi ý phối đồ



luồng gợi ý phối đồ · 6 bước, 1 điểm quyết định

Khi người dùng chưa hài lòng, engine trả bộ khác từ cùng danh sách đã chấm điểm. Khi người dùng chọn một bộ, bộ đó được ghi vào lịch mặc và số lần mặc được làm mới, để lần sau engine ưu tiên món ít mặc.

6. Mô hình dữ liệu

Năm thực thể đủ cho MVP; `Outfit` và `Garment` liên kết nhiều–nhiều qua `OutfitItem`, còn `WearLog` ghi lại mỗi lần mặc.

Thực thể	Trường chính	Ghi chú
User	id, tên, thành phố, phong cách ưa thích	Thành phố dùng để lấy thời tiết
Garment	id, userId, ảnh (đường dẫn cục bộ và URL), loại, màu chính, độ chỉn chu, mùa phù hợp, tag tự do, ngày thêm, lần mặc cuối, số lần mặc	Lần mặc cuối và số lần mặc suy ra từ WearLog, lưu thêm để lọc nhanh

Thực thể	Trường chính	Ghi chú
Outfit	id, userId, tên, hoạt động, yêu thích, nguồn (gợi ý hoặc tự tạo)	Hoạt động: đi học, đi làm, cà phê, hẹn hò
OutfitItem	outfitId, garmentId, vị trí (áo, quần, giày, phụ kiện)	Bảng nối nhiều–nhiều
WearLog	id, userId, outfitId, ngày mặc, hoạt động, đánh giá	Nguồn để tìm món ít mặc

Mỗi bản ghi có `updatedAt` và cờ đã đồng bộ để hỗ trợ cơ chế offline-first.

7. Bảo mật, quyền riêng tư và hiệu năng

Ảnh quần áo và lịch mặc là dữ liệu cá nhân, nên mặc định mọi thứ ở chế độ riêng tư và chỉ chủ tài khoản truy cập được.

Hạng mục	Biện pháp
Xác thực	Firebase Auth; Firestore Security Rules chỉ cho chủ tài khoản đọc và ghi dữ liệu của mình
Lưu trữ ảnh	Cloud Storage chia thư mục theo userId, không công khai
Khóa API của dịch vụ AI	Chỉ năm trong Cloud Functions, không đưa vào app
Quyền thiết bị	Xin camera, thư viện ảnh, thông báo đúng lúc dùng; vị trí là tùy chọn, có thể nhập thành phố thủ công
Dữ liệu người dùng	Có chức năng xóa tài khoản kèm toàn bộ ảnh và lịch mặc; màn hình đồng ý nói rõ ảnh được gửi đến dịch vụ AI để gắn tag
Hiệu năng	Nén ảnh và tạo ảnh thu nhỏ trước khi tải lên; lưới tủ đồ tải theo trang
Offline	Xem tủ đồ, nhận gợi ý và ghi lịch mặc khi mất mạng; thêm đồ mới vào hàng đợi, gắn tag AI khi có mạng lại

8. Rủi ro, đánh đổi và giả định cần xác nhận

Rủi ro lớn nhất là chất lượng gắn tag: tag sai làm gợi ý sai, và người dùng sẽ quay lại bộ quen thuộc, đúng vấn đề Closy muốn giải quyết.

Rủi ro	Cách giảm trong kiến trúc
AI gắn sai loại hoặc màu	Luôn hiện tag để người dùng xác nhận hoặc sửa trước khi lưu
Số hóa tủ đồ tốn công, người dùng bỏ dở	Cho nhận gợi ý ngay khi tủ mới có ít món; thêm đồ theo lô
Engine theo luật cho kết quả lặp lại	Ưu tiên món ít mặc và thêm yếu tố ngẫu nhiên có kiểm soát; kiểm chứng bằng usability testing
Chi phí hoặc giới hạn của API AI	Giới hạn số lần gọi mỗi người dùng, lưu kết quả theo ảnh để không gọi lại
Xung đột đồng bộ giữa hai thiết bị	Giữ bản ghi sửa sau cùng; chấp nhận mất một lần sửa ở trường hợp hiếm
Lộ ảnh cá nhân	Security Rules theo chủ sở hữu, thư mục riêng, chức năng xóa dữ liệu

Cần nhóm xác nhận trước khi nộp

- [ ] Stack thực tế của nhóm có phải Flutter và Firebase không; nếu khác, cập nhật mục 3.
- [ ] MVP có gồm gắn tag bằng AI hay chỉ gắn tag thủ công.
- [ ] Phạm vi nền tảng: chỉ Android hay cả iOS.
- [ ] Đối chiếu với mục MVP Scope, User Stories và Critical User Flows (mục 8–10) của Assignment 1, vì bản đã đính kèm mới hiển thị đến mục 1.3.